

APUNTESWEB · INGENIERÍA INFORMÁTICA



Tercer curso · 1.º cuatrimestre · 7 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Introducción

1.1 Conceptos Principales

1.1.1 Definición de Inteligencia Artificial (IA)

La IA no tiene una única definición, pero se puede entender desde dos perspectivas clave:

- 1. **Definición Institucional (UE):** Sistemas que muestran comportamiento inteligente analizando su entorno y tomando acciones (con cierto grado de autonomía) para lograr objetivos específicos.
- 2. **Definición Funcional:** Un sistema es inteligente si posee **autonomía** significativa, riqueza de comportamiento en entornos dinámicos, capacidad de **aprender de la experiencia** y competencia en áreas especializadas.

1.1.2 Los 4 Enfoques de la IA

Se clasifica la IA según su objetivo final (compararse con humanos o ser idealmente racional) y su faceta (pensamiento o comportamiento):

	Como Personas (Humanista)	Racionalmente (Idealista)
Pensamiento	Modelado Cognitivo (intentar imitar cómo piensa el cerebro).	Leyes del Pensamiento (Lógica formal, silogismos).
Comportamiento	Test de Turing (actuar de forma indistinguible a un humano).	Agentes Racionales (actuar para maximizar el resultado esperado).

El Test de Turing tiene como referencia el comportamiento humano.

1.1.3 La Triada de la IA Moderna

El auge actual de la IA (desde 2010) se debe a la convergencia de tres factores:

- 1. **Algoritmos:** Nuevas técnicas de aprendizaje automático (Deep Learning).
- 2. **Datos:** Disponibilidad masiva de datos (Big Data).
- 3. **Computación:** Capacidad de procesamiento (GPUs/TPUs).

1.2 Explicaciones Paso a Paso

1.2.1 Evolución Histórica: Los "Inviernos" y "Primaveras"

La historia de la IA no es lineal, ha pasado por ciclos de optimismo exagerado y decepción⁵:

1. **Nacimiento (1956):** Conferencia de **Dartmouth**. Se acuña el término "Inteligencia Artificial" por McCarthy, Minsky, et al.⁶⁶⁶⁶.
2. **Entusiasmo (1956-1973):** Predicciones optimistas (ej. el Perceptrón que podría "caminar, hablar y reproducirse")⁷.
3. **Primer Invierno (1974-1980):** Declive por falta de resultados prácticos frente a las promesas.
4. **Sistemas Expertos (1981-1987):** Recuperación comercial.
5. **Segundo Invierno (1988-1993):** Estancamiento.
6. **Relanzamiento y Realismo (1994-Presente):** Éxito basado en datos, aprendizaje profundo y aplicaciones específicas.

1.2.2 Paradigmas: Simbólico vs. Subsimbólico

1. IA Simbólica (Top-Down):

- Se basa en reglas explícitas y lógica y árboles de decisión.
- *Ejemplo:* Un árbol de decisión para saber qué comió un invitado ("Si comió carne -> No es vegetariano").
- Es interpretable por humanos.

2. IA Subsimbólica (Bottom-Up):

- Se basa en **Redes Neuronales**. No se programan reglas, se entrena el sistema con ejemplos.
- Intenta imitar la estructura de las neuronas biológicas.
- *Ejemplo:* Deep Learning para reconocimiento de imágenes.

1.2.3 La Neurona Artificial (Perceptrón)

El componente básico de la IA subsimbólica es la neurona artificial. Matemáticamente se describe así:

$$Y = f \left(\sum_{i=1}^m (W_i \cdot X_i) + b \right)$$

Desglose de componentes:

- X_i (**Inputs**): Las señales de entrada (datos).
- W_i (**Pesos / Weights**): La importancia de cada entrada. Es lo que la red "aprende" durante el entrenamiento.

- Σ (**Función Suma**): Agrega todas las entradas ponderadas.
- b (**Sesgo / Bias**): Un umbral interno de activación (mencionado como "Internal Activation" en el gráfico).
- $f(\cdot)$ (**Función de Activación**): Decide si la neurona se "dispara" o no, introduciendo no linealidad (permite aprender patrones complejos).
- Y (**Output**): El resultado final de la neurona.

NOTA

Esto no rallarse que se explica en los siguientes temas, este tema es para que suenen las cosas no para memorizarlo.

1.3 Conexiones: "Wetware" vs. Hardware

Una comparación crítica para entender por qué la IA es diferente a la inteligencia biológica:

Característica	Cerebro Humano (Wetware)	Computador (Hardware)
Evolución	Biológica (lenta, 100k años sin cambios).	Tecnológica (exponencial, Ley de Moore).
Procesamiento	Paralelo masivo (10 mil millones de neuronas).	Serial (muy rápido en secuencia).
Fortalezas	Percepción, sentido común, aprendizaje general.	Cálculos matemáticos, lógica formal, almacenamiento masivo.
Consumo	Muy eficiente (~20 Watts).	Muy costoso (Megavatios en supercomputadores).
Velocidad	"Lenta" (disparo neuronal en ms).	"Rápida" (ciclos de reloj en nanosegundos).

Conexión clave: La IA actual intenta emular la capacidad de aprendizaje del cerebro (redes neuronales) utilizando la velocidad de cálculo del hardware para compensar la falta de eficiencia biológica.

1.4 Aplicaciones Reales

1. Juegos de Estrategia (Hitos):

- **Deep Blue (1997):** Vence a Kasparov en ajedrez (fuerza bruta y búsqueda).
- **AlphaGo (2016):** Vence a Lee Sedol en Go (aprendizaje profundo y refuerzo, un problema mucho más complejo que el ajedrez).

2. Vehículos Autónomos:

- Uso de sensores complejos: **LIDAR** (láser para mapa 3D), **RADAR** y cámaras.

- Evolución desde el "DARPA Challenge" hasta coches comerciales (Tesla/Volvo)¹⁶¹⁶¹⁶¹⁶.

3. Generación de Lenguaje Natural (NLG):

- **GALIWeather:** Sistema que convierte datos numéricos meteorológicos en predicciones escritas en texto natural (gallego).

4. Robótica:

- **Boston Dynamics:** Robots con equilibrio dinámico y movilidad avanzada (Atlas, Spot)¹⁸.

5. Limitaciones (Predicción Social):

- Caso elecciones EE.UU. 2016: La IA falló al predecir la victoria de Trump basándose solo en "sentimiento en redes", mostrando que el ruido en los datos puede llevar a errores graves¹⁹.

1.5 Repercusiones Socioeconómicas y Éticas de la IA

1.5.1 Contexto: La 4ª Revolución Industrial

La IA no es solo una tecnología aislada, sino el motor de una nueva fase industrial.

- **1ª Revolución:** Vapor y agua (sustitución fuerza animal).
- **2ª Revolución:** Electricidad y producción en masa.
- **3ª Revolución:** Electrónica, internet y automatización simple.
- **4ª Revolución (Actual):** Sistemas Ciber-físicos y **Automatización Inteligente**.

1.5.2 Automatización y Empleo

La Matriz de Automatización

No todos los trabajos se automatizan igual. Se clasifican según la capacidad requerida (Manual vs. Cognitiva) y el tipo de tarea (Sistemática vs. No sistemática):

Tipo de Tarea	Capacidad Manual	Capacidad Cognitiva
Sistemática (Predecible)	Robots de soldadura y montaje (Fábricas)	Sistemas expertos (Análisis de riesgo bancario)
No Sistemática (Impredecible)	Robots de exploración en entornos abiertos	Generación de noticias (Periodismo automatizado)

- *Dato clave:* Las actividades físicas en entornos predecibles tienen un **81%** de potencial de automatización, mientras que la gestión de personas solo un **9%**.

Impacto Real (Ejemplos)

- **Manufactura:** La empresa *Changying Precision Technology* reemplazó al 90% de sus trabajadores con robots, logrando un aumento de producción del 250%.
- **Periodismo:** El "Quakebot" de *Los Angeles Times* genera noticias sobre terremotos automáticamente usando datos del servicio geológico8888.

La Paradoja de la Desigualdad

A medida que avanza la tecnología, se observa una paradoja económica:

- La desigualdad **entre países** disminuye (países en desarrollo crecen).
- La desigualdad **dentro de los países** aumenta (brecha entre trabajadores cualificados/propietarios de capital y trabajadores desplazados).

¿Es esta vez diferente? (Debate Keynesiano)

Keynes predijo el "desempleo tecnológico". Sin embargo, la revolución de la IA presenta desafíos únicos respecto a revoluciones pasadas:

1. **Velocidad:** El proceso se está acelerando.
2. **Alcance:** Afecta a tareas de nivel cognitivo medio y alto, no solo físico.
3. **Reubicación:** Es difícil reubicar a los trabajadores desplazados en nuevos sectores.

1.5.3 Ética y Riesgos Existenciales

Superinteligencia y la "Explosión de Inteligencia"

Concepto propuesto por **I.J. Good (1965)** y analizado por **Nick Bostrom**:

- **Definición:** Una máquina que supera intelectualmente a cualquier humano.
- **Consecuencia:** Esta máquina podría diseñar máquinas aún mejores, provocando una "explosión de inteligencia" que dejaría atrás a la capacidad humana.
- **El Riesgo:** La primera máquina ultrainteligente es el *último* invento que el hombre necesita crear, siempre que sepamos cómo mantenerla bajo control.

Armas Autónomas ("Killer Robots")

Existe un debate ético activo y campañas para detener el desarrollo de armas totalmente autónomas que deciden atacar sin intervención humana (ej. campaña *Stop Killer Robots*).

Privacidad y Responsabilidad Civil

- **Vigilancia:** Uso masivo de reconocimiento facial y cámaras¹⁹.
- **Responsabilidad:** El caso del atropello mortal de un coche autónomo de **Uber** en Arizona plantea quién es el culpable (¿el software, el conductor de seguridad, el fabricante?)²⁰.

1.5.4 Regulación: La "AI Act" (Ley de IA de la UE)

La Unión Europea propone un marco regulatorio basado en niveles de riesgo para los sistemas de IA21:

Nivel de Riesgo	Ejemplos	Implicaciones / Sanciones
Riesgo Inaceptable	Score social, vigilancia biométrica en tiempo real por gobiernos.	Prohibición absoluta. Multas hasta 6% de ingresos globales o 30M€.
Riesgo Alto	Infraestructuras críticas, educación (admisión), justicia, dispositivos médicos.	Evaluación de conformidad obligatoria. Multas hasta 4% o 20M€.
Riesgo Limitado	Deepfakes, Chatbots, sistemas que interactúan con personas.	Obligación de transparencia (avisar que es una IA).
Riesgo Mínimo	Videojuegos, filtros de spam.	Sin restricciones específicas (códigos de conducta voluntarios).

2. Búsqueda en Espacio de Estados

2.1 Conceptos Principales

Para resolver problemas en IA, formalizamos el mundo como un conjunto de estados y transiciones.

1. **Espacio de Estados:** Conjunto de todos los estados alcanzables desde el estado inicial mediante una secuencia de operadores
2. **Estado Inicial:** Descripción de partida del sistema
3. **Operadores (Acciones):** Reglas que transforman un estado en otro (ej. mover una pieza, mover el hueco en un puzzle)
4. **Prueba de Meta (Goal Test):** Condición que determina si un estado es la solución
5. **Costo de Ruta ($g(n)$):** Costo acumulado desde el inicio hasta el estado actual (ej. número de movimientos, distancia en km)

La Función de Evaluación ($f(n)$): Es una fórmula matemática fundamental para guiar la búsqueda, decidiendo qué nodo es el más "prometedor" para expandir a continuación. Asigna un valor numérico a cada nodo que representa la estimación de cuán bueno es ese camino para llegar a la solución. Permite ordenar los nodos en la "frontera" de búsqueda. El algoritmo siempre elegirá expandir el nodo con el mejor valor $f(n)$ (generalmente el menor valor si hablamos de costes).

Componentes típicos:

$$f(n) = g(n) + h(n)$$

- $g(n)$: Lo que ya has recorrido (costo real desde el inicio hasta n)
- $h(n)$: Lo que te falta (estimación heurística desde n hasta la meta)

INFO

Tipos de grafos generados en los procesos de búsqueda:

- **Implícito:** Es la **representación teórica** o virtual del espacio de estados completo. Se define únicamente mediante el **estado inicial** y los **operadores** (reglas de transición), ya que sus nodos se generan dinámicamente bajo demanda y no se almacenan en memoria debido a su tamaño potencialmente infinito.
- **Explícito:** Es el **subgrafo real** que el algoritmo de búsqueda ha generado y **almacenado en la memoria** del ordenador durante su ejecución. Está formado por los nodos que han sido visitados o expandidos (listas ABIERTA y CERRADA) en el intento de encontrar la solución.

2.2 Estrategias de Búsqueda: Paso a Paso

Las estrategias se dividen en **Ciegas** (sin información del dominio) y **Heurísticas** (con información/estimaciones).

Heurística: Es un criterio basado en conocimiento del problema que ayuda a seleccionar los operadores o acciones más prometedoras, descartando opciones poco útiles. **Una heurística es como un "atajo inteligente" que evita búsquedas exhaustivas.**

2.2.1 Búsqueda a Ciegas (No Informada)

Las **estrategias de búsqueda a ciegas** (no informadas) exploran posibles soluciones sin información adicional sobre cuál camino puede ser mejor. Se utilizan principalmente en problemas donde no se tiene una **heurística** clara para guiar la búsqueda.

Características de una búsqueda:

- **Búsqueda completa:** garantiza encontrar una solución si existe.
- **Búsqueda óptima:** garantiza encontrar la mejor solución disponible.
- **Complejidad temporal:** número total de nodos explorados durante la búsqueda.
- **Complejidad espacial:** número máximo de nodos almacenados en memoria simultáneamente

Símbolos:

- **r:** factor de ramificación (promedio de sucesores por nodo)
- **p:** profundidad de la solución
- **m:** profundidad máxima
- **l:** límite de profundidad

Búsqueda en Amplitud (Breadth-First Search - BFS)

Funcionamiento: Explora nivel por nivel. Primero visita el nodo raíz, luego todos sus hijos, luego los nietos, etc.

Estructura de datos: Usa una lista ABIERTA como cola FIFO (Primero en entrar, primero en salir)

Propiedades:

- **Completa:** Sí, encuentra la solución si existe (en espacios finitos)
- **Óptima:** Sí, pero solo si el costo de los operadores es uniforme
- **Problema:** Consume muchísima memoria ($O(r^p)$) porque guarda todos los nodos del nivel actual

Búsqueda en Profundidad (Depth-First Search - DFS)

Funcionamiento: Explora una rama hasta el final antes de retroceder. Si llega a un punto muerto, vuelve atrás.

Estructura de datos: Usa una lista ABIERTA como pila LIFO (Último en entrar, primero en salir)

Propiedades:

- **Completa:** No (puede caer en bucles infinitos o ramas infinitas)
- **Óptima:** No (puede encontrar una solución muy profunda antes que una corta)
- **Ventaja:** Muy eficiente en memoria ($O(r \cdot m)$), solo guarda la rama actual

Otras Búsquedas

- **En profundidad limitada.** Se establece un límite máximo de profundidad para la exploración. Evita ciclos infinitos, pero puede perder soluciones si el límite es bajo.
- **En profundidad iterativa.** Aplica búsqueda en profundidad con límites crecientes, combinando ventajas de amplitud y profundidad. Es completa y óptima (en espacios finitos).
- **Bidireccional.** Realiza la búsqueda simultáneamente desde el estado inicial y desde el objetivo, esperando que ambas se encuentren. Reduce la complejidad temporal y espacial.

Estrategia	Completa	Óptima	Tiempo	Espacio
Amplitud	Sí	Sí	$O(r^p)$	$O(r^p)$
Profundidad	No	No	$O(r^m)$	$O(r * m)$
Prof. limitada	Si cuando $l > p$	No	$O(r^l)$	$O(r * l)$
Iterativa	Sí	Sí	$O(r^p)$	$O(r * p)$
Bidireccional	Sí	Sí	$O(r^{(p/2)})$	$O(r^{(p/2)})$

2.2.2 Búsqueda Heurística (Informada)

A diferencia de la búsqueda a ciegas (que explora sin rumbo), la búsqueda informada utiliza una "pista" o estimación llamada **Heurística** ($h(n)$).

- **¿Qué es una Búsqueda Informada?** Es aquella que tiene conocimiento sobre el problema (como un mapa o una brújula) para estimar cuánto falta para llegar a la meta. Esto le permite **ordenar** las opciones y probar primero las que parecen más prometedoras, en lugar de probarlas al azar.

Antes de ver los algoritmos, debemos distinguir cómo toman decisiones:

Concepto Clave: Irrevocable vs. Tentativa

Tipo de Estrategia	Definición Sencilla	Ejemplo Real
Irrevocable	" Quemar las naves ". Tomas una decisión y avanzas sin guardar el camino de vuelta. No puedes rectificar. Si te equivocas, fallas. Ahorra mucha memoria pero es arriesgada.	Escalar una montaña en la niebla: solo subes por donde está más empinado. Si llegas a un pico falso, no puedes bajar y buscar otro pico.
Tentativa	" Dejar migas de pan ". Guardas información de las alternativas no exploradas. Si un camino no funciona, puedes volver atrás (rectificar) y probar otro.	Explorar un laberinto con un hilo atado a la entrada. Si hay un muro, regresas por el hilo y pruebas otro pasillo.

Búsqueda Retroactiva (Backtracking)

Tipo: **Tentativa** (Permite rectificar)

Es una estrategia inteligente que intenta no gastar memoria. Avanza en profundidad, pero si se equivoca, vuelve sobre sus pasos.

- **Funcionamiento:**

- Elige el mejor hijo disponible (usando la heurística para ordenar) y avanza.
- **Gestión de memoria estricta:** Solo recuerda el **camino actual** (la ruta activa desde el inicio), no todo el árbol.
- **Vuelta atrás:** Si llega a un "callejón sin salida" (punto muerto), retrocede al padre y prueba el siguiente hijo prometedor 2.

- **Utilidad:** Ideal cuando tienes muy poca memoria pero necesitas encontrar una solución, aunque no sea la más corta.

NOTA

Si $f(n) = g(n)$ es una búsqueda no informada, el backtracking no se si el profesor lo considera o no como búsqueda heurística. Yo sobreentiendo que no.

Algoritmo de Escalada (Hill Climbing/Primeiro o melhor)

Tipo: **Irrevocable** (Versión estricta del algoritmo ávaro)

Es el ejemplo clásico de estrategia irrevocable. Busca la cima de la montaña dando siempre el paso que más sube, sin mirar atrás.

- **Estrategia:** Evalúa los vecinos y se mueve *inmediatamente* al que tiene mejor heurística (parece estar más cerca de la meta), **descartando y olvidando el resto**.

- **Función de evaluación:** $f(n) = h(n)$ (Solo importa lo que falta).
- **Riesgo:** Como no guarda la historia ni alternativas, puede quedarse atrapado en **máximos locales** (una pequeña colina que parece la cima pero no lo es) o **mesetas** (donde todos los pasos son iguales y no sabe a dónde ir).

Algoritmo A* (A-Estrella)

Tipo: Tentativa La más completa es la estrategia estrella porque equilibra la precaución con la eficiencia. Guarda alternativas (es tentativa) y evalúa el coste total.

- **Estrategia:** No solo mira cuán cerca parece estar la meta (h), sino cuánto le ha costado llegar hasta ahí (g).
- Función de evaluación:

$$f(n) = g(n) + h(n)$$

(Costo Total = Costo ya pagado + Costo estimado restante).

- **Propiedad Clave:** Si la heurística es **admisible** (nunca es pesimista, es decir, $h(n) \leq$ coste real), A* garantiza encontrar la solución **óptima** (la más barata/corta) y es **completa** (siempre encuentra solución, sea admisible o no la heurística).

INFO

Una heurística admisible u optimista: siempre suponer casos en los que la realidad será peor que la estimación.

Si eres Pesimista ($h >$ coste real): Imagina que hay un atajo secreto que tarda 15 minutos. Pero tu heurística es pesimista y le dice al algoritmo: "Por ese callejón vas a tardar 1 hora".

- **Consecuencia:** El algoritmo se asusta, ve un coste altísimo y **no explora ese camino**.
- **Resultado:** Pierdes la oportunidad de encontrar el mejor camino (la solución óptima) porque tu estimación te engañó diciendo que era malo.

Si eres Optimista ($h \leq$ coste real): Tu heurística le dice al algoritmo: "Por ese callejón parece que tardas solo 5 minutos".

- **Consecuencia:** Como parece un camino barato, el algoritmo **lo explora**.
- **Resultado:** Al entrar, se da cuenta de que en realidad son 15 minutos (la realidad es peor que la estimación, como dice tu nota), pero **al menos lo exploró**.
- A* prefiere equivocarse pensando que un camino es "demasiado bueno" (porque así lo comprueba) que equivocarse pensando que es "demasiado malo" (porque entonces lo ignora y podría ser la solución).

En el caso del 8-puzzle se acostumbra a usar estas **heurísticas: Misplaced Tiles (Piezas Descolocadas)**: Es la más simple. Simplemente **cuentas cuántas fichas no están en su posición final**.

- **Cálculo:** Si la ficha '1' está en la casilla del '2', suma 1. Si la '2' está bien colocada, suma 0.
- **Ejemplo:** Si 5 fichas están mal puestas, $h(n) = 5$

Distancia de Hamming: En el contexto del 8-puzzle, es **sinónimo de "Misplaced Tiles"**.

- Proviene de la teoría de la información (número de posiciones en las que dos cadenas de símbolos difieren). Aquí compara el "estado actual" con el "estado meta" y cuenta las diferencias.

Distancia de Manhattan (City Block): Es más precisa (y más informada) que la anterior.

- **Cálculo:** Para cada ficha, calculas cuántas casillas debe moverse (horizontal + vertical) para llegar a su destino. Luego **sumas todas esas distancias**.
- **Por qué se llama así:** Porque simula moverse por las calles de Manhattan (en cuadrícula), no puedes ir en diagonal.
- *Ejemplo:* Si la ficha '1' está a 2 casillas a la derecha y 1 abajo de su meta, su distancia es 3.

Heurística de Gaschnig: es una **relajación** del problema. Asume que puedes mover cualquier ficha a la posición del hueco (teletransportándola), no solo las adyacentes.

- **Cálculo:** Cuenta el número de intercambios necesarios para resolver el puzle si pudieras intercambiar el hueco con **cualquier** ficha del tablero para ponerla en su sitio de un solo movimiento.
- Suele dar un valor entre la de Hamming y la real.

2.3 Caso de Estudio Práctico: Ruta en Ciudades Gallegas

Este caso ilustra la diferencia entre ser "miope" (Voraz) y ser "inteligente" (A^*).

El Problema: Ir de Ferrol a Ourense

- $g(n)$: Distancia real por carretera (tramos negros en el mapa)
- $h(n)$: Distancia en línea recta a Ourense (números rojos en paréntesis)

A. Ejecución Búsqueda Voraz ($f = h$)

Solo mira la distancia recta a la meta.

1. Ferrol va a Betanzos
2. Betanzos tiene vecinos: Santiago ($h = 80$) y Guitiriz ($h = 70$)
3. **Decisión:** Elige Guitiriz porque $70 < 80$ (parece más cerca)

4. **Resultado:** Termina encontrando una ruta de 240 km. **No es la mejor.**

B. Ejecución Búsqueda A* ($f = g + h$)

Mira el pasado (g) y el futuro (h).

1. Ferrol va a Betanzos ($g = 50$)
2. En Betanzos, evalúa opciones:
 - Ruta por Guitiriz: $g(50 + 70) + h(70) = 120 + 70 = 190$
 - Ruta por Santiago: $g(50 + 50) + h(80) = 100 + 80 = 180$
3. **Decisión:** Elige Santiago porque $180 < 190$. Aunque Santiago parece más lejos en línea recta, el costo total estimado es menor.
4. **Resultado:** Encuentra la ruta óptima de 230 km pasando por Santiago y Silleda.

2.4 Destellos de calidad del Senén en clase

2.4.1 Problema del Viajante de Comercio (TSP)

- **Concepto:** Es un problema clásico de optimización combinatoria. Dado un conjunto de nodos (ciudades) y las distancias entre ellos, el objetivo es encontrar un **ciclo hamiltoniano** de coste mínimo. Esto significa hallar una ruta cerrada que visite cada nodo exactamente una vez y regrese al nodo de origen, minimizando la distancia total recorrida.
- **Ejemplo:** Un repartidor debe visitar **100 ciudades**. Parte de una central fija (ciudad 0). Debe trazar una ruta que pase por las 99 restantes una sola vez y vuelva a la 0, gastando la menor cantidad de combustible posible.

2.4.2 Tamaño del Espacio de Búsqueda

- **Concepto:** Es la cardinalidad del conjunto de todas las **soluciones posibles** (válidas) para el problema, independientemente de si son óptimas o no. En problemas de permutación como el TSP, este tamaño crece de forma factorial con respecto al número de elementos variables.
- **Ejemplo:** En el caso de 100 ciudades con un punto de partida y final fijo (la ciudad de origen no se permuta), el número total de rutas posibles son las permutaciones de las 99 ciudades restantes.

$$\text{Tamaño} = 99! \quad (\text{Factorial de } 99)$$

2.4.3 Tamaño del Entorno (Contorna)

- **Concepto:** El entorno es el subconjunto de soluciones vecinas accesibles desde la solución actual aplicando **una única vez un operador** de transformación específico. Su tamaño (N) depende del operador elegido y del número de elementos sobre los que se puede aplicar.

- Ejemplo: Usando el operador de intercambio de dos elementos (swap) sobre las 99 ciudades móviles (excluyendo origen/final).

Debemos calcular cuántas combinaciones de 2 elementos distintos se pueden formar con los 99 disponibles. Matemáticamente, son combinaciones sin repetición de 99 elementos tomados de 2 en 2:

$$N = \binom{99}{2} = \frac{99 \times 98}{2}$$

2.4.4 Búsqueda Tabú

- **Concepto:** Es una **metaheurística de búsqueda local** diseñada para escapar de óptimos locales. Explora el entorno de la solución actual y se mueve a la mejor vecina (aunque sea peor que la actual), pero utiliza una estructura de **memoria a corto plazo** (lista tabú) para restringir la búsqueda.
- **Ejemplo:** Si el algoritmo decide intercambiar la ciudad A con la B para generar una nueva ruta, este movimiento se registra en la lista tabú. Durante un número determinado de iteraciones (tenencia tabú), el algoritmo tiene **prohibido** deshacer ese cambio (volver a intercambiar B con A), forzando así la exploración de nuevas zonas del espacio de búsqueda en lugar de entrar en ciclos repetitivos

3. Sistemas Basados en Conocimiento

3.1 Introducción y Definición

A diferencia de los sistemas de "búsqueda en espacio de estados" (vistos anteriormente), donde se busca una solución explorando estados ciegamente o con heurísticas, los SBC se basan en **conocimiento explícito** para razonar.

Diferencias Clave: Búsqueda vs. Conocimiento.

Característica	Búsqueda en Espacios de Estados	Sistemas Basados en Conocimiento
Base	Estados y operadores	Base de Conocimiento (Reglas/Lógica)
Método	Algoritmo de búsqueda (exploración)	Método de razonamiento (inferencia)
Datos Iniciales	Estado inicial	Hechos conocidos
Resultado	Camino o estado solución	Respuesta a una consulta / Diagnóstico

3.2 Sistemas Expertos (SE)

Son el tipo más representativo de SBC. Se definen como sistemas con un nivel de competencia equivalente o superior a un experto humano en un dominio concreto.

¿Cuándo usar un Sistema Experto? No sirven para todo. Se recomiendan cuando:

- El problema es complejo y de un dominio especializado.
- **No existe un algoritmo tradicional** para resolverlo.
- Existen expertos humanos capaces de articular ese conocimiento.

Historia y Ejemplos

- **Dendral (1965):** Identificación de compuestos químicos.
- **Mycin (años 70):** Diagnóstico de infecciones bacterianas.

3.3 Arquitectura de un Sistema Experto

Para que una máquina "razone", necesita separar lo que *sabe* (conocimiento) de lo que *está pasando* (hechos) y de *cómo pensar* (motor).

Componentes Principales:

1. **Base de Conocimiento (BC):** Es estática y general del dominio. Contiene las reglas ("Si X entonces Y").
2. **Base de Hechos (BH):** Es dinámica y específica del caso actual. Contiene los datos del problema y lo que se va descubriendo.
3. **Motor de Inferencia:** Es el "cerebro". Aplica el conocimiento a los hechos para deducir nuevas verdades. Realiza tareas de **equiparación** (matching) y **resolución de conflictos**.

Hechos	Conocimiento
Específicos, relacionados con el problema concreto que se intenta resolver.	General, relacionado con el dominio en el que se opera y el tipo de problemas.
Dinámicos, a partir de unos hechos iniciales pueden aparecer otros distintos.	Relativamente estático, aunque puede cambiar si se añade nuevo conocimiento.
Aumenta durante la resolución del problema.	Normalmente no aumenta durante la resolución del problema.
Necesidad de almacenamiento y recuperación eficiente.	Necesidad de razonamiento eficiente.
Se busca que los hechos obtenidos directamente del problema sean precisos y exactos.	Puede ser impreciso e incierto; por lo tanto, también los hechos inferidos pueden serlo.

3.4 Representación del Conocimiento

¿Cómo guardamos el conocimiento en la máquina?

Lógica Formal (La base matemática)

Es la forma más rigurosa de representar relaciones.

- **Lógica Proposicional:** Usa afirmaciones verdaderas/falsas y operadores (AND, OR, NOT, Implicación). Se basa en reglas de inferencia. Imagina siempre una regla condicional: $P \rightarrow Q$ ("Si pasa P, entonces pasa Q"):
 - **Modus Ponens:** Si la regla es cierta ($P \rightarrow Q$) y la condición (P) se cumple, entonces la consecuencia (Q) **también es cierta**.
 - **Modus Tollens.** Si la regla es cierta ($P \rightarrow Q$), pero vemos que la consecuencia (Q) **NO** ha ocurrido, entonces la causa (P) **tampoco pudo ocurrir**.
- **Lógica de Predicados:**
 1. **Objetos (Constantes):** Las cosas del mundo real (ej: Juan, Ana, Coche).
 2. **Predicados:** Son las **propiedades** o **relaciones** que afectan a los objetos. Se escriben como funciones:

- `es_hombre(Juan)` → Predicado de propiedad.
- `es_hermano(Juan, Ana)` → Predicado de relación.

3. **Variables:** Usamos letras (x, y) para referirnos a "cualquier cosa" sin especificar quién.

4. **Cuantificadores:** Las herramientas poderosas:

- **Universal ($\forall$ - "Para todo"):** Permite crear reglas generales.
 - *Ejemplo:* $\forall x$ (Si x es humano $\rightarrow x$ es mortal).
- **Existencial ($\exists$ - "Existe"):** Indica que hay al menos uno.
 - *Ejemplo:* $\exists x$ (Padre(x , Juan)) $\rightarrow$ "Existe alguien que es el padre de Juan".

Reglas de Producción y Sistemas Basados en Reglas (SBR)

Es la estructura más utilizada en los Sistemas Expertos. Cuando representamos el conocimiento de esta forma, hablamos de **Sistemas Basados en Reglas**.

- **Unidad básica:** La Regla de Producción.
 - Formato: `SI <condición/situación> ENTONCES <acción>`.
- **Dinámica del SBR:**
 - La parte `SI` busca coincidencias en la **Base de Hechos**.
 - La parte `ENTONCES` modifica la memoria: añade nuevos hechos, borra antiguos o ejecuta acciones.
- **Ejemplo:**
 - *Regla:* SI "coche no arranca" Y "batería < 10V" ENTONCES "cambiar batería".

Redes Semánticas

Representación gráfica mediante grafos.

- **Nodos** = Conceptos/Objetos (ej. "Juan", "Persona").
- **Arcos** = Relaciones o herencia (ej. "es un", "tipo de"). Permite heredar propiedades (si Juan es Persona, y Persona tiene altura, Juan tiene altura).

3.5 Estrategias de Razonamiento (El Motor en acción)

Una vez tenemos reglas y hechos, ¿cómo los procesa el motor? Existen dos estrategias opuestas:

Encadenamiento Hacia Adelante (Forward Chaining)

- **Filosofía:** Guiado por los **datos** (Data-driven).

- **Proceso:** Partimos de los hechos conocidos → buscamos reglas que se cumplan → ejecutamos acciones (disparamos reglas) → obtenemos nuevos hechos hasta llegar a un objetivo o no poder seguir .
- **Ejemplo (Coches):** Sé que "el coche no arranca" y "batería < 10V" → Deduzco "cambiar batería".
- **Algoritmo:** Ciclo de **Equiparar** (ver qué reglas se cumplen) → **Resolver** (elegir una del conjunto conflicto) → **Aplicar** (ejecutarla).

El **conjunto conflicto** es el grupo de reglas que, en un momento determinado del razonamiento, **cumplen todas sus condiciones** (antecedentes) en base a los hechos disponibles y **aún no fueron ejecutadas**.

Es decir, no es una sola regla, sino la colección de **todas las reglas posibles** que el sistema *podría* ejecutar en ese instante porque sus requisitos ("SI...") son ciertos.

El **principio de refracción** evita la aplicación reiterada de una **regla aplicable**. Imagina que no existiera este principio. Tienes la siguiente situación:

- **Hecho:** (temperatura 40)
- **Regla:** SI temperatura > 38 ENTONCES imprime "Tiene Fiebre"

¿Qué pasaría sin refracción?

1. **Ciclo 1:** El motor ve que la temperatura es 40. Se cumple la regla. **Acción:** Imprime "Tiene Fiebre".
2. **Ciclo 2:** El motor vuelve a mirar los hechos. La temperatura *sigue* siendo 40 (nadie la ha borrado). La regla se cumple de nuevo. **Acción:** Imprime "Tiene Fiebre".
3. **Ciclo 3:** La temperatura sigue siendo 40... **Acción:** Imprime "Tiene Fiebre".

El sistema entraría en un **bucle infinito**, repitiendo la misma acción una y otra vez, porque la condición sigue siendo verdad. Para que no pase, el motor de inferencia dice: *"Vale, esta regla se cumple con el dato (temperatura 40), PERO yo ya ejecuté esta regla exacta con este dato exacto en el ciclo anterior. Así que no la voy a volver a meter en el conjunto conflicto hasta que los datos cambien."*

Encadenamiento Hacia Atrás (Backward Chaining)

- **Filosofía:** Guiado por los **objetivos** (Goal-driven).
- **Proceso:** Partimos de una **hipótesis** (¿Tiene gripe?) → buscamos reglas que concluyan eso → verificamos si se cumplen sus premisas (antecedentes).
- **Estructura:** Se suele modelar como un **Grafo AND/OR**. Para probar una meta, debo probar sus sub-metas.
- **Ejemplo (Médico):** ¿Tiene infección? (Meta) → Para ello necesito saber si tiene fiebre y dolor (Sub-metas) → Verifico temperatura.

3.6 Comparativa: SE vs. Programación Convencional

Es fundamental entender que programar un Sistema Experto no es programar un algoritmo clásico.

Programación Convencional	Sistemas Expertos
Imperativa (Cómo hacerlo)	Declarativa (Qué se sabe)
Modificación por re-programación de código	Modificación añadiendo/quitando reglas en la base de conocimiento
Solución algorítmica (precisa, única)	Solución inferida (puede ser incierta/probabilística)
Guiada por flujo de control	Guiada por el Motor de Inferencia

Tendencias Modernas: LLMs y "Chain of Thought":

- Los Grandes Modelos de Lenguaje (LLMs) pueden realizar razonamientos lógicos (silogismos), aunque a veces fallan si las premisas del mundo real entran en conflicto con la lógica formal (ej. "todos los pájaros vuelan" vs "pingüinos").
- **Chain-of-Thought (CoT):** Técnica de *prompting* donde se pide al modelo que "piense paso a paso". Esto mejora drásticamente la capacidad de resolver problemas matemáticos o lógicos complejos en comparación con pedir solo la respuesta final.

3.7 CLIPS (C Language Integrated Production System)

¿Qué es CLIPS?

Es una herramienta de software diseñada para desarrollar **Sistemas Expertos** y sistemas basados en reglas. Funciona mediante un paradigma de **programación declarativa**: tú dices *qué* sabes (hechos) y *qué* hacer si ocurren ciertas cosas (reglas), y el motor decide *cuándo* hacerlo.

Los 3 Pilares de la Memoria

Debes distinguir claramente entre estos tres espacios:

1. **Base de Conocimiento (Knowledge Base):** Es donde se almacena la "inteligencia" estática del programa. Aquí viven las Reglas (`defrule`), las Plantillas (`deftemplate`) y las definiciones de hechos iniciales (`defacts`).
 - *Se llena usando:* El comando `(load)`.
2. **Memoria de Trabajo (Working Memory):** Es la "pizarra" volátil donde están los datos actuales (los Hechos). Cambia constantemente.
 - *Se llena usando:* Los comandos `(reset)` y `(assert)`.

3. **Motor de Inferencia:** El "cerebro" que compara constantemente la *Memoria de Trabajo* con la *Base de Conocimiento* para ver qué reglas activar.

Los Hechos (Datos)

1. Tipos de Hechos

- **Vectores Ordenados:** Listas simples. `(Pedro 45 V)`. Rígidlos.
- **Plantillas (Deftemplates):** Estructurados con campos. `(Persona (Nombre Juan) (Edad 30))`. Flexibles.

2. Propiedades Críticas para el Examen (IMPORTANTE)

a) El Hecho Especial `f-0` (initial-fact)

- **¿Qué es?** Es el "hecho cero". El primer dato que CLIPS crea automáticamente cuando reinicias el sistema.
- **¿Para qué sirve?** Actúa como una chispa de arranque. Permite que se disparen reglas que no tienen condiciones específicas en su parte izquierda (reglas que quieres que se ejecuten "siempre" al principio).
- **Prohibición:** El usuario **NO puede definir ni manipular el hecho f-0**. Está reservado. Si intentas hacer `(assert (f-0 ...))` o borrarlo manualmente sin cuidado, el sistema puede fallar o ignorarte.

b) Unicidad (Conjuntos)

- La memoria de CLIPS es un **conjunto matemático**, no una lista.
- **No hay duplicados:** Si intentas añadir un hecho idéntico a uno que ya existe (mismos campos, mismos valores), CLIPS **no hace nada**. No crea un duplicado ni genera un nuevo ID. Simplemente mantiene el viejo.

El Ciclo de Vida: De Cargar a Ejecutar

Aquí es donde suelen pillar en las preguntas tipo test (como la pregunta 14 de tu imagen). El orden lógico es:

1. `(clear)`: El Borrado Total

- **Acción:** Elimina **TODO**. Deja el sistema vacío, como recién instalado.
- **Efecto:** Borra hechos, reglas, templates y deffacts.

2. `(load "archivo.clp")`: La Carga

- **Acción:** Lee un archivo de texto `.clp` línea por línea.
- **Efecto (OJO):**

- Carga las definiciones (`defrule`, `deftemplate`, `deffacts`) en la **Base de Conocimiento**.
- **NO ejecuta nada**.
- **NO mete hechos en la Memoria de Trabajo** (a menos que haya `asserts` sueltos fuera de reglas, lo cual es mala práctica).
- Los hechos definidos en `deffacts` se quedan "en espera", memorizados, pero aún no activos.

3. `(reset)` : El Reinicio y Preparación

Este comando es el puente entre la teoría (lo cargado) y la práctica (la memoria). Realiza 3 pasos estrictos:

1. **Borra** todos los hechos actuales de la Memoria de Trabajo (`retract *`).
2. **Crea** automáticamente el hecho `f-0` (**initial-fact**).
3. **Activa** todos los hechos que estaban definidos en los `deffacts` que cargaste previamente con `load`, metiéndolos ahora sí en la Memoria de Trabajo.

4. `(run)` : La Ejecución

- **Acción:** Arranca el Motor de Inferencia.
- **Efecto:** El motor mira los hechos que `reset` acaba de poner, busca reglas que coincidan (Pattern Matching), las mete en la Agenda y las ejecuta hasta que no queden más

La Agenda y el Orden de Ejecución (MUY IMPORTANTE)

La Agenda es la lista de tareas pendientes del sistema.

¿Qué hay dentro de la Agenda?

No contiene solo reglas, ni solo hechos. Contiene **Activaciones (Instancias)**.

- Definición de Activación: Es la pareja formada por:

$$\text{Regla} + \text{HechosEspecíficos}(IDs)$$

- *Ejemplo:* Si tienes la regla "Si tienes hambre, come" y el hecho "tengo hambre", en la agenda aparece: `Activación: Regla_Comer + Hecho_f-1`.

¿Cómo decide CLIPS qué ejecutar primero? (Resolución de Conflictos)

Si hay varias reglas en la agenda (Conjunto Conflicto), CLIPS tiene que elegir una. El orden por defecto suele ser **Salience + LIFO (Depth Strategy)**.

A. Salience (Prioridad Explícita)

- Puedes dar "rangos militares" a las reglas.
- `(declare (salience 100))`: Esta regla es VIP. Se ejecutará antes que una de salience 0.

- Rango: de -10,000 a +10,000.

B. Recency (Recencia - LIFO)

- Si dos reglas tienen la misma prioridad (saliencia), ¿cuál va primero?
- **Regla de oro:** CLIPS favorece a los hechos **más recientes**.
- *Comportamiento de Pila (Stack):* El último hecho que entró (`assert`) es el primero que dispara reglas. Esto permite que el sistema se centre en "lo último que ha pasado" antes de volver a tareas antiguas.

3. Casos que se pueden dar en la Agenda

Caso 1: Agenda Vacía

- Si haces (`run`) y no hay activaciones, el programa termina inmediatamente. No hace nada.

Caso 2: Una regla coincide con varios hechos distintos

- Imagina la regla: `Si (número ?n) => imprimir ?n.`
- Hechos: (`número 1`) y (`número 2`).
- **Resultado:** En la agenda habrá **dos activaciones** distintas de la misma regla. La regla se ejecutará dos veces (una para el 1 y otra para el 2).

Caso 3: Principio de Refracción (Loop Prevention)

- **Pregunta de examen:** ¿Se ejecuta una regla infinitamente si el hecho sigue ahí?
- **Respuesta: NO.** CLIPS recuerda que ya ejecutó la regla `R1` con el hecho `f-1`. Aunque `f-1` siga existiendo, esa activación específica se borra de la agenda para no repetirse eternamente. Solo se volverá a activar si modificas el hecho (porque al modificarlo cambia su ID o su "timestamp").

Ejemplo Práctico de Traza

Imagina que tienes este archivo `enfermedades.clp`: Fragmento de código

```
; DEFINICIÓN DE DATOS INICIALES (La lista de la compra)
(deffacts mis-datos
  (temperatura 40)
  (garganta_inflamada)
)

; REGLA
(defrule detectar-fiebre
  (temperatura ?t > ?t 38)
  =>
```

```
(printout t "Tienes fiebre" crLF)
)
```

Secuencia de Comandos:

1. (load "enfermedades.clp"):

- CLIPS lee el archivo. Memoriza que existe una regla `detectar-fiebre` y un grupo de datos `mis-datos`.
- *Estado Memoria Trabajo*: **VACÍA**. (Ni siquiera `f-0`).

2. (reset):

- Paso 1: Borra lo que hubiera (no borra reglas)
- Paso 2: Crea `f-0 (initial-fact)`.
- Paso 3: Lee el `deffacts mis-datos` y hace assert de `f-1 (temperatura 40)` y `f-2 (garganta_inflamada)`.
- *Estado Memoria Trabajo*: `f-0, f-1, f-2`.

3. (run):

- El motor ve `f-1 (temperatura 40)`.
- La regla `detectar-fiebre` se activa (porque $40 > 38$).
- Sale por pantalla: "Tienes fiebre".

4. Sistemas Conexionistas

4.1 Introducción: Del Símbolo a la Neurona

A diferencia de los sistemas expertos (donde un humano programa las reglas), los sistemas conexionistas se inspiran en la estructura física del cerebro para **aprender** esas reglas a partir de ejemplos.

- **Dualidad:** Combinan la **Bioinspiración** (imitan el cerebro) con la **Tecnoinspiración** (modelos matemáticos para el cálculo) .
- **Evolución Histórica:**
 - 1943: Modelo de McCulloch & Pitts (lógica binaria).
 - 1957: El **Perceptrón** de Rosenblatt (primera regla de aprendizaje).
 - 1969: Minsky y Papert demuestran las limitaciones (problema XOR), provocando un "invierno de la IA".
 - 1986: Renacer con el algoritmo de Retropropagación (Backpropagation) para redes multicapa 2.

4.2 La Neurona Artificial: Unidad Básica

Al igual que en biología, la neurona es la unidad de procesamiento. Existe una analogía directa entre las partes biológicas y el modelo matemático:

Biología	Modelo Matemático (Artificial)	Función
Dendritas	Entradas (x_i)	Reciben señales de otras neuronas o del exterior.
Sinapsis	Pesos (w_i)	Determinan la importancia/fuerza de cada entrada.
Soma	Función de suma ($\sum$)	Agrega todas las señales ponderadas.
Axón	Salida (y)	Transmite el resultado si se supera un umbral.

Modelo Matemático

Una neurona calcula una suma ponderada de sus entradas más un sesgo (bias), y pasa el resultado por una función de activación.

$$z = \sum_i x_i \cdot w_i + b$$

- x_i : Señales de entrada.

- w_i : Pesos sinápticos (el "conocimiento" de la red se almacena aquí).
- b (**Bias**): Umbral de activación (matemáticamente equivale a un peso extra w_0 con entrada fija +1).

Funciones de Activación (φ)

Determinan la salida final de la neurona:

1. **Escalón (Binaria)**: Salida 1 si supera el umbral, 0 si no. (Modelo original). Es una decisión rígida.
2. **Lineal Rectificada (ReLU)**: Si $z < 0$ sale 0; si $z \geq 0$ sale z ($y = \max(0, z)$). Actúa como un filtro de "pase". Si la señal es relevante (positiva), la deja pasar tal cual; si es irrelevante (negativa), la anula por completo.
3. **Sigmoidal**: Transforma la salida en un valor suave entre 0 y 1 (probabilidad). Fórmula: $y = \frac{1}{1+e^{-z}}$. En lugar de decir "Es un gato" (1) o "No es un gato" (0), la neurona dice "Hay un 85% de probabilidad de que sea un gato" (0.85).

DANGER

Cuidado con la **terminología**, en función de la función de activación que se use, cambia el nombre de la neurona.

Nombre	Función de Activación (f(z))	Salida	Interpretación	Regresor
Lineal	Identidad (Ninguna)			

$f(z) = z$ | Números Reales

$(-\infty, +\infty)$ | **Predicción**

"La casa vale 250.000€" | **Regresor Logístico** | **Sigmoide**

$f(z) = \frac{1}{1+e^{-z}}$ | Probabilidad

$(0, 1)$ | **Clasificación Suave**

"Hay un 85% de probabilidad de que sea spam" | **Perceptrón** | **Escalón** (Step)

$f(z) = 1$ si $z > 0$ | Binaria

$\{0, 1\}$ | **Clasificación Dura**

"Es spam (1). Punto."

Se aplican **al final del procesamiento de la neurona**, justo después de calcular la suma ponderada y antes de enviar la señal a la siguiente capa.

El proceso paso a paso es:

1. **Recepción:** La neurona recibe las entradas $(x_1, x_2 \dots)$ multiplicadas por sus pesos $(w_1, w_2 \dots)$.
2. Suma (z): Se suman todos estos valores más el sesgo (b). Esto ocurre en la "unión sumadora" (Σ).

$$z = \sum x_i w_i + b$$

3. Activación (φ): Aquí es donde se aplica la función. El valor z entra en la "caja" de la función de activación y se transforma en la salida final y .

$$y = \varphi(z)$$

4.3 Análisis Geométrico: ¿Qué "ve" una neurona?

Una sola neurona funciona como un **clasificador lineal**.

- **Frontera de decisión:** La neurona divide el espacio en dos zonas (ej. Clase A y Clase B).
- Matemáticamente, esta frontera es una recta (o hiperplano) definida por:

$$x_1 \cdot w_1 + x_2 \cdot w_2 + b = 0$$

- **Interpretación:**
 - A un lado de la recta, la suma es positiva $\rightarrow$ La neurona se activa ($y = 1$).
 - Al otro lado, la suma es negativa $\rightarrow$ La neurona se apaga ($y = 0$).

Nota de estudio: Los pesos w definen la inclinación de la recta y el bias b define su desplazamiento respecto al origen.

En la parte superior ves escritas las "instrucciones" o la configuración de esta neurona específica:

- $w_1 = 2$: El peso de la entrada 1.
- $w_2 = 2$: El peso de la entrada 2.
- $b = 1$: El bias (sesgo).

La neurona calcula una suma: $(x_1 \cdot w_1) + (x_2 \cdot w_2) + b$.

La frontera (la línea dibujada) es el punto exacto donde esa suma da 0. Es el límite entre dispararse o no. Si sustituimos los valores en la fórmula:

$$2x_1 + 2x_2 + 1 = 0$$

Para poder dibujarla en un gráfico de ejes X e Y (donde x_2 actúa como la Y vertical y x_1 como la X horizontal), despejamos x_2 :

1. Pasamos el resto al otro lado: $2x_2 = -2x_1 - 1$
2. Dividimos todo por 2: $x_2 = -x_1 - \frac{1}{2}$

¡Mira la esquina inferior derecha de la imagen! Ahí está escrita exactamente esa fórmula final: $x_2 = -\frac{1}{2} - x_1$ (aunque el orden está invertido, es lo mismo).

Aquí es donde ves el efecto del **bias**.

- **Si el bias fuera 0:** La línea pasaría exactamente por el cruce de los ejes (el origen).
- **Como el bias es 1:** La ecuación nos dice que la línea corta el eje vertical en $-1/2$ (o -0.5).
- **Visualmente:** Puedes ver en el dibujo que la línea diagonal **no pasa por el centro**, sino que está desplazada hacia abajo y a la izquierda. Ese desplazamiento es culpa del bias.

La línea divide el papel en dos territorios:

- Zona + $\rightarrow$ zona de $y = 1$: Es todo el espacio que queda arriba y a la derecha de la línea. Cualquier punto que caiga aquí (por ejemplo, el punto $(1, 1)$) hará que la suma sea positiva (> 0).
 - *Interpretación:* La neurona se **activa** (dispara un 1).
- Zona - $\rightarrow$ zona de $y = 0$:
Es el espacio que queda abajo y a la izquierda (marcado con una flecha y un signo menos). Cualquier punto aquí (por ejemplo, $(-2, -2)$) hará que la suma sea negativa (< 0).
 - *Interpretación:* La neurona se **inhibe** (se queda en 0).

4.4 Arquitecturas de Red: ¿Cómo organizamos las neuronas?

No basta con tener neuronas; la clave es cómo las conectamos entre sí. Dependiendo de las conexiones, la red sirve para cosas muy distintas.

4.4.1 Redes Monocapa (Perceptrón Simple)

Es la forma más básica.

- **Estructura:** Tienes una capa de entradas y una capa de salida. Las entradas se conectan directamente a las salidas.
- **Flujo:** La información va en un solo sentido (hacia adelante).
- **Limitación:** Como vimos en el apartado 4.6, estas redes **solo pueden resolver problemas lineales**. Si los datos no se pueden separar con una línea recta (como el problema XOR), esta red falla.
- **Matemáticamente:** Es simplemente $Salida = Función(Entrada \cdot Peso)$.

4.4.2 Redes Multicapa (Feedforward)

Aquí es donde la IA se pone interesante.

- **Estructura:** Entre la entrada y la salida, metemos capas intermedias llamadas **Capas Ocultas**.
- **¿Por qué "Ocultas"?** Se llaman así no porque "no se vean en el diseño" (eso es falso), sino porque no tienen conexión directa con el mundo exterior (ni con los datos de entrada brutos ni con la respuesta final).
- **Función:** Rompen la linealidad.
 - La primera capa detecta patrones simples (bordes).
 - La capa oculta combina esos patrones simples para crear formas complejas (curvas, esquinas).
 - Esto permite resolver problemas **no lineales** (como el XOR, explicado al final).

4.4.3. Redes Recurrentes (ej. Redes de Hopfield)

- **Estructura:** Tienes bucles. La salida de una neurona puede volver hacia atrás y conectarse a la entrada de sí misma o de otras neuronas anteriores.
- **Superpoder: La Memoria.** Al tener bucles, la información "da vueltas" y persiste en el tiempo.
- **Uso:** Son vitales para datos secuenciales (texto, audio, series temporales) porque la red recuerda lo que pasó en el instante anterior.

4.5 El Aprendizaje (Algoritmo del Perceptrón)

¿Cómo aprende la red? Ajustando sus pesos (w) iterativamente basándose en el error cometido. Es un aprendizaje **supervisado** (necesitamos ejemplos con la respuesta correcta).

ACORDASE DE QUE PERCEPTRÓN -> NEURONA ARTIFICIAL

Algoritmo de Convergencia del Perceptrón:

1. **Inicialización:** Poner los pesos a 0.
2. **Activación:** Presentar un ejemplo $x(n)$ y calcular la respuesta real $y(n)$.
3. Adaptación de Pesos (Regla de aprendizaje):

$$w(n+1) = w(n) + \eta \cdot [d(n) - y(n)] \cdot x(n)$$

- $w(n)$: Peso actual.
- η : **Tasa de aprendizaje** (cuánto cambiamos el peso en cada paso).
- $d(n)$: Salida deseada (correcta).
- $y(n)$: Salida obtenida (real).

Lógica del algoritmo:

- Si la red acierta ($d = y$), el término $[d - y]$ es 0 $\rightarrow$ No se cambian los pesos.
- Si falla, los pesos se ajustan en la dirección del error para corregirlo la próxima vez.
- η : debe ser muy pequeño para no provocar cambios significativos en cada iteración, pero no tanto como para evitar o realentizar la convergencia.

4.6 Limitaciones y Soluciones: El Problema XOR

Problemas Linealmente Separables

Limitación del Perceptrón Simple: Solo puede clasificar conjuntos **linealmente separables** (aquellos que se pueden separar con una sola línea recta). Son los problemas "fáciles" para una neurona.

- **¿Por qué son lineales?** Porque una sola neurona (Perceptrón Simple) funciona matemáticamente dibujando **una línea recta** (como vimos en la imagen anterior: $x_1 \cdot w_1 + x_2 \cdot w_2 + b = 0$).
- **Ejemplo Clásico (AND y OR):**
 - Imagínate la función lógica **AND** (Y). Solo es "verdad" (1) si ambas entradas son 1.
 - Si pintas esto en una gráfica, el punto (1,1) es el único "positivo". Los otros tres (0,0), (0,1), (1,0) son "negativos".
 - **Prueba visual:** Puedes dibujar una línea que deje al (1,1) en una esquina y a los otros tres en el otro lado 1.

Problemas No Linealmente Separables (El caso XOR)

Son los problemas "difíciles" donde una sola neurona falla estrepitosamente.

- **¿Por qué son no lineales?** Porque la estructura de los datos es compleja. Una sola recta no basta.
- **El Ejemplo Maldito: XOR (O Exclusivo).**
 - La regla del XOR es: "Es verdad (1) si una es verdadera y la otra falsa. Si son iguales, es falso (0)".
 - **Puntos Positivos (1):** (0,1) y (1,0).
 - **Puntos Negativos (0):** (0,0) y (1,1).
 - **Prueba visual:** Dibuja estos cuatro puntos en un papel. Intenta separar los positivos de los negativos con **una sola línea recta**. ¡Es imposible! Si separas uno, dejas al otro en el lado equivocado. Siempre te dejas uno fuera .
- **¿Qué pasa si intentas usar un Perceptrón Simple?**
 - El algoritmo de aprendizaje nunca termina (entra en bucle) o se queda con un error muy alto, moviendo la línea desesperadamente de un lado a otro sin encontrar solución .

¿Cómo se soluciona lo "No Lineal"?

Si una sola línea (una neurona) no puede separar los datos, ¿qué haces? **Usas más líneas.**

Aquí es donde entran las **Redes Neuronales Multicapa**:

1. **Capa Oculta:** Usas dos neuronas para dibujar **dos rectas distintas**. Una recta separa un grupo y la otra recta separa el otro.
2. **Capa de Salida:** Combina la información de esas dos rectas para dar la respuesta final.

Observando la imagen de arriba: La Parte Izquierda: Aquí vemos el espacio de entrada (x_1 vs x_2) con 4 puntos:

- **Signos Menos Rojos (-):** Están en $(0,0)$ y $(1,1)$. Son los casos donde la salida debe ser **0** (Falso).
- **Cruces Verdes (+):** Están en $(0,1)$ y $(1,0)$. Son los casos donde la salida debe ser **1** (Verdadero).

El Problema: Como ves, es imposible dibujar una sola línea recta que deje a todas las cruces verdes a un lado y a los signos rojos al otro.

La Solución: La imagen muestra dos líneas (una naranja y una azul).

- La línea **naranja** separa el rojo de abajo $(0,0)$.
- La línea **azul** separa el rojo de arriba $(1,1)$.
- La zona "válida" (Verde) es la franja que queda **entre medias** de las dos líneas.

La Parte Derecha: Aquí vemos cómo se construye esa "franja" usando neuronas. Es una red con una **capa oculta** de dos neuronas (h_1 y h_2).

- **Neurona Naranja (h_1):**
 - Fíjate en su ecuación: $20x_1 + 20x_2 - 10$.
 - Esta neurona es la responsable de dibujar la **línea naranja** del gráfico. Se activa si la suma de las entradas es suficiente para superar el primer umbral. Básicamente dice: "*¿Estamos por encima de (0,0)?*".
- **Neurona Azul (h_2):**
 - Fíjate en su ecuación: $-20x_1 - 20x_2 + 30$ (pesos negativos).
 - Esta neurona dibuja la **línea azul**. Se activa si las entradas **no** son demasiado grandes. Básicamente dice: "*¿Estamos por debajo de (1,1)?*".
- **Neurona de Salida (y):**
 - Recibe las señales de la Naranja y la Azul.
 - Su trabajo es hacer una operación **AND**: Solo se activa si la Naranja dice "Sí" (estamos encima de la primera línea) **Y** la Azul dice "Sí" (estamos debajo de la segunda línea).

4.7 Neurona de BIAS

Matemáticamente, el bias es el término b en la ecuación de una recta:

$$y = m \cdot x + b$$

En una neurona, la fórmula es:

$$z = (w_1 \cdot x_1 + \dots) + \mathbf{b}$$

Sirve para **desplazar** la función de activación hacia la izquierda o la derecha. En teoría, la fórmula es *Sumatoria* + b . Pero en programación (y álgebra lineal), sumar un número suelto b al final de una multiplicación de matrices es molesto computacionalmente. Preferimos que todo sean multiplicaciones ordenadas.

Para arreglar esto, usamos un **truco arquitectónico**: Inventamos una "Neurona de Bias".

- **Qué es:** Es una neurona de entrada extra que añadimos artificialmente a la capa.3
- **Su valor:** Siempre vale **1**. Nunca cambia.
- **Su peso:** El peso (w_0) que conecta esta neurona falsa con la siguiente capa **es el valor del bias**.

Estas neuronas no se cuentan como el resto de neuronas, por si preguntan en el test el número de neuronas de una red.

5. Aprendizaje Automático (Machine Learning)

5.1 Introducción y Cambio de Paradigma

Hasta ahora (Sistemas Expertos), programábamos la IA diciéndole explícitamente *qué hacer* (reglas **SI-ENTONCES**). En el Aprendizaje Automático, cambiamos el enfoque: **la máquina aprende las reglas a partir de los datos**.

Un programa aprende si mejora su rendimiento (**P**) en una tarea (**T**) basándose en la experiencia (**E**).

- **Ejemplo (Damas):**
 - **T (Tarea):** Jugar a las damas.
 - **E (Experiencia):** Jugar miles de partidas contra sí mismo.
 - **P (Rendimiento):** % de partidas ganadas.

5.2 Tipos de Aprendizaje

No todos los problemas son iguales. Según la información que tengamos, clasificamos el aprendizaje en:

5.2.1 Aprendizaje Supervisado

Tenemos los "exámenes resueltos". Entregamos a la máquina datos de entrada (x) junto con su respuesta correcta o etiqueta (y) (por ello se les dice **datos etiquetados**). El objetivo es aprender la relación para predecir y en datos nuevos.

Se divide en dos subtipos clave:

1. **Regresión:** Predice un **valor numérico continuo** (infinitas posibilidades).
 - *Ejemplo:* Predecir el precio de una casa según sus metros cuadrados.
2. **Clasificación:** Predice una **clase o etiqueta discreta** (categorías fijas).
 - *Ejemplo:* ¿Es este tumor maligno o benigno? (0 o 1).
 - *Nota:* Esto es lo que hacía el Perceptrón (clasificar).

5.2.2 Aprendizaje No Supervisado

Solo tenemos los datos (x), sin respuestas. La máquina debe encontrar **estructuras o patrones** por sí sola. trabaja con **datos no etiquetados**.

- *Ejemplo (Clustering)*: Agrupar noticias similares en Google News o segmentar clientes por comportamiento .

5.3 Regresión Lineal: El Modelo Base

Es el "Hola Mundo" del Machine Learning. Busca dibujar la **línea recta** que mejor se ajusta a una nube de puntos.

5.3.1 La Hipótesis (h_θ)

Es la fórmula matemática que usamos para predecir. En regresión lineal simple (una variable), es la ecuación de una recta:

$$h_\theta(x) = \theta_0 + \theta_1 x$$

- θ_0 : El punto de corte (Bias).
- θ_1 : La pendiente (Peso).

Conexión Clave: ¿Te suena? Es idéntico a la parte interna de una neurona ($z = w \cdot x + b$).

- Por esto, en el examen, un **Regresor Lineal** es lo mismo que una **Neurona Artificial con función de activación Lineal (Identidad)**.
- Si a esa suma no le pones función de activación (o la función es $f(x) = x$), la neurona se comporta exactamente como este modelo de regresión.

5.3.2 La Función de Coste ($J(\theta)$)

Para aprender, necesitamos medir cuánto nos equivocamos. Usamos el Error Cuadrático Medio (MSE):

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum (h_\theta(x^{(i)}) - y^{(i)})^2$$

- Básicamente: "Calcula la diferencia entre lo que predijiste y el valor real, elévalo al cuadrado (para que sea siempre positivo) y haz la media".
- **Objetivo:** Encontrar los θ (pesos) que hagan que esta J sea **mínima** (cero error idealmente).
- La función de coste de un **regresor lineal** mide lo bien o mal que opera el regresor lineal en un conjunto de datos. Es una función convexa.
- Esto es una función de coste convexa.

5.3.3 Generalización: Regresión Lineal Multivariable

El modelo básico ($h_{\theta}(x) = \theta_0 + \theta_1 x$) solo sirve si hay una única característica de entrada. Pero el mundo es complejo.

- **Concepto:** si tenemos n características ($x_1, x_2, \dots, x_n$), la fórmula se expande
- **Nueva Hipótesis:**

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

- **Vectorización:** Para que el ordenador lo calcule rápido, esto se expresa como una multiplicación de matrices: $h_{\theta}(x) = \theta^T x$.

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}, x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{metros} \\ \text{habitaciones} \end{bmatrix}$$

$$\text{Si } \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}, \text{ entonces } \theta^T = [\theta_0, \theta_1, \theta_2].$$

$$\theta^T x = [\theta_0, \theta_1, \theta_2] \times \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2$$

5.3.4 Adaptación a Curvas: Regresión Polinómica

A veces, los datos no siguen una línea recta, sino una curva.

- **El Truco:** no necesitamos inventar un algoritmo nuevo. Podemos "engañar" a la regresión lineal creando **nuevas características artificiales** elevando las originales al cuadrado, al cubo, etc.
- **Hipótesis:**

$$\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2$$

- **Peligro:** Al añadir términos polinómicos ($x^2, x^3 \dots$), la línea se vuelve muy flexible y curva. Esto mejora el ajuste, pero si nos pasamos, causaremos **Overfitting** (esto se explica más adelante).

Imagina que quieres predecir cuánto vas a gastar en electricidad (y) basándote en la temperatura de la calle (x).

- **Intento con Regresión Lineal (Línea Recta):**
 - La máquina pensaría: "*Cuanto más calor hace, más gastas*" (Línea subiendo) O BIEN "*Cuanto más calor hace, menos gastas*" (Línea bajando).
 - **¿Por qué falla?** Porque la realidad es más compleja:
 - Si hace mucho frío (0°C), gastas mucho (calefacción).

- Si hace una temperatura media (20°C), gastas poco (ni frío ni calor).
 - Si hace mucho calor (40°C), vuelves a gastar mucho (aire acondicionado).
- **Forma de los datos:** Tienen forma de "U" (una parábola).
- **Solución (Regresión Polinómica):**
 - Al añadir el término al cuadrado (x^2), la fórmula puede curvarse y crear esa "U".
 - **Conclusión:** El modelo aprende que el gasto sube en *ambos extremos* de la temperatura.

NOTA

En un regresor polinomial, cuanto mayor es el grado del polinomio, mejor podemos ajustar el modelo a su conjunto de entrenamiento. No implica que vaya a funcionar mejor el modelo, seguramente haya alto overfitting.

5.4 Regresión Logística (Para Clasificación)

5.4.1 La hipótesis

(Importante: Aunque se llame "regresión", sirve para **clasificar**). Es el modelo matemático que conecta la Regresión Lineal con las Neuronas Artificiales.

- **El Problema:** La regresión lineal predice valores continuos (puede dar 500, -20, 1000). Pero si queremos clasificar (0 o 1, Tumor benigno o maligno), necesitamos que la salida esté acotada entre 0 y 1.
- **La Solución (Función Sigmoide):** Envolvemos la hipótesis lineal dentro de una función logística o sigmoide ($g(z)$).

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

- **Interpretación:** La salida se convierte en una **probabilidad**.
 - Si $h_{\theta}(x) = 0.7$, significa "Hay un 70% de probabilidad de que sea clase 1".
- **Frontera de Decisión:** Es la línea (o curva) que separa la zona donde predecimos 0 de la zona donde predecimos 1. Se produce cuando la probabilidad es exactamente 0.5.

5.4.2 Función de Coste

Se define en dos partes:

- Si la respuesta real es 1 ($y = 1$):

$$Coste = -\log(h_{\theta}(x))$$

- *Interpretación:* Si la máquina predice 1 (acierta), el coste es 0. Si predice 0 (falla estrepitosamente), el coste tiende a infinito (castigo enorme).

- Si la respuesta real es 0 ($y = 0$):

$$Coste = -\log(1 - h_{\theta}(x))$$

- *Interpretación:* Si predice 0 (acierta), coste 0. Si predice 1 (falla), coste infinito.

Para no escribir dos fórmulas, las combinamos en una sola ecuación elegante:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

- *Nota:* Solo uno de los dos términos se activa a la vez (porque y es 0 o 1, anulando la otra parte).

5.4.3 Clasificación Binaria vs. Multiclase

La Regresión Logística está diseñada por naturaleza para responder "Sí/No" (Binaria). ¿Pero qué pasa si quiero clasificar correos en "Trabajo", "Amigos" y "Spam" (3 clases)?

A. Clasificación Binaria (2 Clases)

- **Datos:** $y \in \{0, 1\}$.
- **Frontera de Decisión:** Una sola línea (o curva) que separa el espacio en dos zonas.
- **Modelo:** Una sola hipótesis $h_{\theta}(x)$.

B. Clasificación Multiclase (Más de 2 grupos)

- **Datos:** $y \in \{0, 1, 2, \dots, n\}$.
- **Estrategia:** Usamos la técnica **"Uno contra Todos" (One-vs-All)**.

En lugar de entrenar un solo clasificador complejo, entrenamos varios clasificadores binarios:

1. Transformamos el problema:

- Imagina que tienes 3 clases: Triángulos, Cuadrados y Círculos.
- Entrenas el **Clasificador 1**: ¿Es un Triángulo o NO? (Juntas cuadrados y círculos en el grupo "NO").
- Entrenas el **Clasificador 2**: ¿Es un Cuadrado o NO?
- Entrenas el **Clasificador 3**: ¿Es un Círculo o NO?

2. Predicción:

- Cuando llega un dato nuevo, lo pasas por los 3 clasificadores.
- Cada uno te dará una probabilidad (ej. C1: 20%, C2: 80%, C3: 10%).

- Eliges la clase con la probabilidad más alta ($\max h_{\theta}^{(i)}(x)$).

5.4.4 Ejemplo

Quieres que la IA decida si un email entrante es "Spam" o "Correo Normal".

- **Intento con Regresión Lineal:**

- Le das al sistema el número de veces que aparece la palabra "Oferta".
- La regresión lineal te podría devolver un valor como 150 o -20.
- **¿Por qué falla?** ¿Qué significa que un correo sea "150 de Spam"? No tiene sentido. Tú necesitas una probabilidad entre 0 y 1.

- **Solución (Regresión Logística):**

- La función **Sigmoide** "aplasta" cualquier número que salga para que esté siempre entre **0 y 1**.
- **Resultado:** Te dice "La probabilidad de que esto sea Spam es 0.95".
- **Decisión:** Como $0.95 > 0.5$ (umbral), lo manda a la carpeta de Spam.

5.5 El Motor de Aprendizaje: Descenso del Gradiente

¿Cómo encuentra la máquina los mejores θ sin probar a lo loco? Usando un algoritmo de optimización iterativo, similar a "bajar una montaña a ciegas tanteando el terreno".

Algoritmo: Repetir hasta converger:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

- **Derivada ($\frac{\partial}{\partial \theta}$):** Nos dice la pendiente. Si es positiva, debemos bajar (restar); si es negativa, subir. Nos da la **dirección** correcta.
- **Tasa de Aprendizaje (α):** Es el **tamaño del paso**.
 - Si α es muy pequeño: El aprendizaje es lentísimo (pasitos de bebé).
 - Si α es muy grande: Podemos "saltarnos" el mínimo y nunca converger (pasos de gigante).
- **Conexión:** Este es el mismo principio matemático que usaba el Perceptrón para actualizar sus pesos ($w_{nuevo} = w_{viejo} + \Delta w$).

NOTA

Normalizar los datos favorece a la convergencia del método, permitiendo que la forma de la función sea más uniforme.

En modelos simples, como el perceptrón, el regresor lineal (también polinómico y multivariable) y el regresor logístico la función de coste es convexa y converge, como en la imagen.

NOTA

Tras revisar el examen del 2021 el profesor considera que la función de coste no tiene por qué ser ni convexa ni cóncava, el motivo ni puta idea supongo que usará un error distinto al MSE. Es bastante loco. Vale tras hacer el examen nos confirmó que cuando hace la pregunta de si es convexa no se refiere a un tipo de función de coste concreta, así que no la marqueis.

En el descenso por gradiente en modelos no lineales, como las redes neuronales, no asegura alcanzar el mínimo global ya que puede quedares en un mínimo local. Por lo tanto la inicialización de los parámetros (los pesos) es importante ya que determina el punto inicial del descenso por gradiente. Tal y como se ve en esta imagen.

Ecuación Normal (Alternativa al descenso gradiente)

Es una forma alternativa de encontrar el valor óptimo de θ . La diferencia fundamental está en **cómo** llegan a la solución:

- **Descenso del Gradiente (Iterativo):** Empiezas con valores aleatorios y vas ajustando poco a poco (iteraciones) hasta minimizar el error. Necesitas elegir un "learning rate" (α).
- **Ecuación Normal (Analítica):** Usas álgebra lineal para resolver el problema de golpe. Es como tener una fórmula mágica que te "teletransporta" directamente al punto más bajo de la función de coste, sin dar pasos. Encuentra la solución exacta matemáticamente.

$$\theta = (X^T X)^{-1} X^T y$$

Aunque parece intimidante, la lógica es similar a resolver una ecuación simple como $5x = 10$. Para despejar x , dividirías por 5 (o multiplicarías por el inverso 5^{-1}). En el mundo de las matrices:

1. Queremos encontrar θ tal que $X\theta \approx y$
2. No podemos simplemente "dividir" por la matriz X .
3. La operación $(X^T X)^{-1} X^T$ es, en términos muy simplificados, la forma matricial de "pasar la X al otro lado dividiendo" para despejar la θ .

Característica	Descenso del Gradiente	Ecuación Normal (La de tu imagen)
Learning Rate (α)	Necesitas elegir uno y ajustarlo.	No necesitas elegir nada.
Iteraciones	Muchas iteraciones.	Ninguna. Es un solo cálculo directo.
Complejidad	Funciona bien incluso con millones de características (n grande). $O(kn^2)$	Se vuelve muy lenta si tienes muchas características (p.ej., $n > 10,000$) porque calcular la inversa de una

Característica	Descenso del Gradiente	Ecuación Normal (La de tu imagen)
		matriz $(X^T X)^{-1}$ es costoso computacionalmente. $O(n^3)$
Resultado	Aproximado (muy cercano al óptimo).	Exacto (el óptimo matemático). Aunque a veces puede no funcionar porque no toda matriz es invertible.

5.6 Problemas Fundamentales: Sesgo vs. Varianza

Cuando entrenamos un modelo, podemos fallar por dos extremos opuestos:

5.6.1 Underfitting (Subajuste / Alto Sesgo)

Ocurre cuando el modelo es **demasiado simple** e ignora la información de los datos porque "ya cree saber la respuesta"

- *Ejemplo*: Intentar ajustar una línea recta a datos que forman una curva parabólica.
- *Síntoma*: Tiene mucho error tanto en el entrenamiento como en la prueba. No aprende bien.

5.6.2 Overfitting (Sobreajuste / Alta Varianza)

Ocurre cuando el modelo es **demasiado complejo** y sensible. Presta tanta atención a los detalles y al ruido de los datos de entrenamiento que, si le cambias un poco los datos, cambia totalmente su predicción. **No aprende, memoriza.**

- *Ejemplo*: Un polinomio de grado 10 que pasa por *todos* los puntos haciendo zig-zag.
- *Síntoma*: Error bajísimo en entrenamiento, pero altísimo en datos nuevos (falla al generalizar).

La primera línea representa la regresión lineal y la segunda la logística.

5.7 Solución: Regularización

¿Cómo evitamos el Overfitting si queremos usar modelos complejos (muchas variables)?

La **Regularización** consiste en "castigar" al modelo si usa pesos (θ) muy grandes. Modificamos la Función de Coste añadiendo un término extra.

Imagina una función matemática (un polinomio) que hace muchas curvas locas para pasar por todos los puntos (Overfitting). Para que una curva suba y baje bruscamente en distancias cortas, necesita multiplicar las variables por números muy grandes (pesos θ altos).

- Si obligamos a que esos números (θ) sean **pequeños**, la curva se vuelve más **suave, plana y sencilla**, evitando esos zig-zags extremos.

La función de coste $J(\theta)$ es lo que el ordenador intenta minimizar (hacer lo más pequeña posible).

$$J(\theta) = \underbrace{\text{Error de Predicción}}_{\text{"Qué tan bien acierto"}} + \underbrace{\lambda \sum \theta_j^2}_{\text{"Impuesto por complejidad"}}$$
$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\Theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^m \theta_j^2 \right]$$

Imagina que eres un profesor evaluando a un alumno (el modelo):

1. **Error Original:** Es la nota del examen. Quieres que falle lo menos posible.
2. **Término de Regularización ($\lambda \sum \theta^2$):** Es una multa por usar "chuletas" o respuestas demasiado complicadas.

El papel de Lambda (λ): El tamaño de la multa. El parámetro λ es el "juez" que decide cuánto nos importa la simplicidad frente a la precisión exacta:

- Si λ es muy ALTO (Multa enorme): El modelo dice: "¡Oye, usar pesos grandes me sale carísimo en la fórmula! Mejor pongo casi todos los pesos (θ) cercanos a cero para que el coste total sea bajo".
 - *Consecuencia:* El modelo se vuelve una línea casi plana. Es **demasiado simple** (riesgo de *Underfitting*).
- Si λ es 0 (Sin multa):
El modelo dice: "Solo me importa acertar los puntos, no me importa si uso ecuaciones complicadísimas".
 - *Consecuencia:* El modelo hace zig-zags locos para tocar todos los puntos. Es **demasiado complejo** (riesgo de *Overfitting*).

La regularización busca el equilibrio perfecto (un λ intermedio) donde el modelo acierte lo suficiente (bajo error) pero manteniendo una curva suave (pesos bajos) para poder generalizar bien con datos nuevos. Reducir varianza, atenúa el efecto de la correlación entre predictores y minimiza la influencia en el modelo de los predictores menos relevantes. Por lo general, aplicando regularización se consiguen modelos con mayor poder predictivo (generalización).

6. Aprendizaje no Supervisado

6.1 Cambio de Paradigma: De la Etiqueta a la Estructura

En la **Regresión Lineal y Logística** (temas anteriores), siempre teníamos un conjunto de datos con la respuesta correcta: $\{(x^{(1)}, y^{(1)}), \dots\}$. La máquina aprendía comparando su predicción con esa y .

En el **Aprendizaje No Supervisado**, la situación cambia radicalmente:

- **Datos:** Solo tenemos x (Datos sin etiquetar). No hay y .
- **Objetivo:** El algoritmo debe encontrar **estructuras**, patrones o agrupamientos en los datos por sí mismo.
- **Rol del Humano:** Es el diseñador quien, a posteriori, le da significado a esos grupos (ej. "Este grupo son clientes VIP", "Este grupo son clientes en riesgo").

6.2 El Algoritmo K-Medias (K-Means)

Es el algoritmo más popular y sencillo para resolver problemas de **Agrupamiento (Clustering)**. Su objetivo es dividir los datos en K grupos (clusters).

Es un proceso iterativo ("bucle") que funciona como un baile de centros de gravedad.

1. Inicialización:

- Decidimos cuántos grupos queremos (K).
- Elegimos aleatoriamente K puntos del mapa para que sean los **Centroides** iniciales ($\mu_1, \mu_2, \dots, \mu_K$).

2. Bucle (Repetir hasta converger):

- **Paso A: Asignación de grupos:** Para cada dato $(x^{(i)})$, calculamos la distancia a todos los centroides y lo "pintamos" del color del centroide más cercano.

$$c^{(i)} := \text{índice del centroide más cercano a } x^{(i)}$$

- **Paso B: Movimiento de Centroides:** Calculamos la media (promedio) de todos los puntos que pertenecen a un grupo y movemos el centroide a esa nueva posición central.

$$\mu_k := \text{promedio de los puntos asignados al grupo } k$$

El algoritmo se detiene cuando los centroides ya no se mueven (convergencia).

6.3 La Función de Coste (Distorsión)

Al igual que en la regresión teníamos el error cuadrático, aquí necesitamos medir "qué tan mal" están agrupados los datos. A esta función se le llama Función de Distorsión (J):

$$J(c, \mu) = \frac{1}{m} \sum_{i=1}^m ||x^{(i)} - \mu_{c^{(i)}}||^2$$

- **Significado:** Mide la suma de las distancias al cuadrado entre cada punto y el centroide de su grupo.
- **Objetivo:** Queremos minimizar J . Si J es bajo, significa que los puntos están muy "apretaditos" alrededor de su centroide (buen agrupamiento).

6.4 Problemas y Soluciones

6.4.1 El Problema de los Mínimos Locales

Como los centroides iniciales se eligen **al azar**, a veces tenemos mala suerte.

- **Riesgo:** Los centroides pueden quedarse "atascados" en una mala posición (Mínimo Local) y no encontrar la mejor agrupación posible (Óptimo Global).
- **Solución:** No ejecutes el algoritmo una sola vez.
 1. Ejecuta K-Medias muchas veces (ej. 50 o 100 veces) con inicializaciones aleatorias diferentes.
 2. Calcula la Distorsión J final para cada intento.
 3. Quédate con la solución que tenga la **menor distorsión**.

6.4.2 ¿Cómo elegir el número de grupos (K)?

A veces el número de grupos es obvio, pero otras veces no. ¿Son 3 grupos o 4?

1. Método del Codo (Elbow Method):

- Ejecutas el algoritmo variando K (ej. $K = 1, 2, 3, 4, 5 \dots$).
- Graficas la función de coste J vs. K .
- Al aumentar los grupos, el error siempre baja. Pero buscamos el punto donde la curva hace un **codo** (deja de bajar rápido y empieza a bajar lento). Ese es el K óptimo.

2. Propósito del Mercado (Market Purpose):

- A veces el "Codo" no es claro. En ese caso, el K lo dicta el negocio.
- *Ejemplo (Tallas de Camisetas):* Tienes datos de altura y peso. Podrías hacer 3 grupos (S, M, L) o 5 grupos (XS, S, M, L, XL). La decisión depende de tu estrategia de ventas, no solo de

la matemática.

6.5 Resumen de Conexiones

Para tu esquema mental global:

- **Regresión Lineal:** Predice un número (Supervisado).
- **Regresión Logística:** Predice una clase binaria (Supervisado).
- **K-Medias:** Descubre grupos sin etiquetas (No Supervisado). Usa distancias geométricas (como los vecinos cercanos) y medias iterativas.

7. Aprendizaje en Redes Multicapa, Retropropagación

7.1 El Problema: El Juego de la Culpa

En un **Perceptrón Simple** (una sola neurona), si el sistema falla, sabemos que la culpa es de esa única neurona. El error es fácil de calcular: `Objetivo - Salida`.

En una **Red Multicapa** (Deep Learning), tenemos un problema grave: **El Problema de Asignación de Crédito**.

- Si la red dice "Gato" y era "Perro", la neurona de salida se equivocó.
- Pero, ¿por qué se equivocó? Quizás porque la neurona oculta de la capa anterior le pasó un dato malo.
- ¿Y por qué esa neurona oculta le pasó un dato malo? Quizás porque la anterior a ella falló.

No tenemos un "profesor" para las capas intermedias. **La Retropropagación** es el método matemático para enviar la "culpa" del error desde el final hacia atrás, capa por capa, para saber cuánto ajustar cada peso interno.

7.2 Definiciones Previas: Anatomía de la Señal

Para entender las fórmulas, primero debemos distinguir qué pasa dentro de cada neurona.

1. Entrada Neta (Net o z): Es la suma bruta de lo que recibe la neurona.

$$Net_j = \sum (x_i \cdot w_{ij})$$

- Es el "ruido total" antes de filtrar.

2. Salida (y o Out): Es el resultado procesado tras pasar el filtro (sigmoide).

$$y_j = f(Net_j)$$

- Es lo que la neurona "grita" a la siguiente capa.

IMPORTANTE

Para poder usar Retropropagación, necesitamos una función de activación que sea **suave y continua** (derivable en todo punto). Usamos la **Sigmoide**:

$$f(z) = \frac{1}{1 + e^{-z}}$$

Esta función curva suavemente la entrada, transformando cualquier número (del $-\infty$ al $+\infty$) en un valor entre **0 y 1**.

Lo mejor de la Sigmoide no es solo su forma, sino que su derivada es increíblemente fácil de calcular en un ordenador. La derivada se puede expresar en función de la propia salida de la neurona (y o $f(z)$):

$$f'(z) = f(z) \cdot (1 - f(z))$$

Para calcular la pendiente (sensibilidad) de la neurona, no necesitas volver a hacer cálculos complejos con exponenciales (e^{-z}). Si la neurona ya sabe su salida (y), simplemente calculas $y \cdot (1 - y)$.

3. **Objetivo (t):** El valor correcto que *deberíamos* haber obtenido (solo existe al final de la red).

7.3 El Fundamento Matemático: El Descenso del Gradiente

Nuestro objetivo es minimizar el **Error Global (E)**. Definimos el error cuadrático medio:

$$E = \frac{1}{2}(t - y)^2$$

- $\frac{1}{2}$: Es un truco matemático. Al derivar x^2 , el 2 baja ($2x$). Al multiplicarlo por $1/2$, se cancelan. Nos simplifica la vida.
- **Derivada**: Para aprender, necesitamos saber la pendiente. Si cambio el peso w , ¿el error sube o baja?

La Necesidad de la Derivada (La Regla de la Cadena)

Queremos cambiar un peso w que está en el medio de la red. Pero ese peso no toca el Error (E) directamente.

1. El peso w cambia la **Entrada Neta (Net)**.
2. La Entrada Neta cambia la **Salida (y)**.
3. La Salida cambia el **Error (E)**.

Por la Regla de la Cadena, la derivada total es la multiplicación de estos pasos:

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial Net} \cdot \frac{\partial Net}{\partial w}$$

Aquí es donde nace el concepto de **DELTA** (δ). Para no escribir todo eso, llamamos δ (Delta o Error Local) a la parte que combina el Error y la Activación: "**Cuánto contribuye la neurona al error total**".

7.4 El Algoritmo: Ciclo de Dos Pasos

El aprendizaje ocurre repitiendo estos dos pasos.

Paso 1: Propagación Hacia Adelante (Forward Pass)

La red funciona normalmente.

1. Metemos los datos en la entrada.
2. Calculamos **Entradas Netas** y **Salidas** capa por capa hasta el final.
3. Comparamos la salida final con el objetivo real.

Paso 2: Propagación Hacia Atrás (Backward Pass) - Calculando Deltas

Ahora calculamos el error local (δ) de cada neurona.

A. Para la Neurona de Salida (Sabemos la respuesta): Aquí es fácil. La culpa es la diferencia directa con la realidad, multiplicada por la sensibilidad de la neurona (su derivada).

$$\delta_{salida} = (t - y) \cdot f'(Net)$$

- $(t - y)$: El error bruto.
- $f'(Net)$: La derivada de la función sigmoide. Significa "¿Qué tan fácil era hacer cambiar de opinión a esta neurona?".

B. Para las Neuronas Ocultas (La Magia): No tenemos $(t - y)$. Calculamos la culpa "escuchando las quejas" de las neuronas de la siguiente capa a las que alimentamos.

$$\delta_{oculta} = f'(Net) \cdot \sum (\delta_{siguiente} \cdot w_{conexion})$$

- $\sum (\delta \cdot w)$: Sumo las culpas de las neuronas siguientes ponderadas por mi conexión con ellas. "Si la neurona siguiente tiene un error gigante (δ alto) y yo le grité muy fuerte (w alto), yo tengo mucha culpa".

NOTA

Esta imagen no se si está bien pero bueno es un poco para entender como se monta todo este tinglao.

7.5 Actualización de Pesos (El Aprendizaje)

Una vez que cada neurona tiene su "nota de culpa" (δ), actualizamos los pesos para corregir el error.

$$w_{nuevo} = w_{actual} + \Delta w$$

La corrección (Δw) se calcula así:

$$\Delta w = \eta \cdot \delta \cdot \text{Entrada}_{origen}$$

Desglose de la fórmula:

1. η (**Tasa de aprendizaje**): La **Prudencia**. "Aunque el error sea grande, corregimos poco a poco para no pasarnos".
2. δ (**Delta del destino**): La **Dirección del Error**. "¿Debo subir o bajar el peso?".
3. Entrada_{origen} (**Salida de la anterior**): La **Evidencia**. "Solo corregimos el peso si la neurona de origen estaba activa. Si envió un 0, este peso no tuvo nada que ver en el resultado, así que no se toca".

7.6 Relación entre todas las fórmulas

Partimos de la fórmula del inicio

$$\frac{\partial E}{\partial w} = \underbrace{\frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial Net}}_{\text{Parte A}} \cdot \underbrace{\frac{\partial Net}{\partial w}}_{\text{Parte B}}$$

Si te fijas en la **Parte A** (los dos primeros términos), verás que coinciden exactamente con la definición de δ_{salida} :

- **1º Término:** $\frac{\partial E}{\partial y}$
 - Como calculamos antes, esto es igual a $-(t - y)$.
 - (El signo menos se suele absorber en la fórmula final de actualización de pesos).
- **2º Término:** $\frac{\partial y}{\partial Net}$
 - Esto es la derivada de la función de activación, es decir, $f'(Net)$.

Si multiplicas esos dos términos, obtienes exactamente tu fórmula de delta:

$$\delta_{salida} \approx \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial Net} = (t - y) \cdot f'(Net)$$

Nota técnica: Matemáticamente, δ se define como $-\frac{\partial E}{\partial Net}$. El signo negativo del error $(y - t)$ se cancela con el menos de la definición, quedando el positivo $(t - y)$.

La **Parte B** ($\frac{\partial Net}{\partial w}$) es lo que nos faltaba, esta derivada es simplemente la entrada x .

Gracias a esta relación, podemos reescribir la terrorífica Regla de la Cadena (primera fórmula) de una forma súper simple y elegante para programar:

$$\frac{\partial E}{\partial w} = -\delta \cdot x_{entrada}$$

Y de ahí sale la famosa regla de actualización:

$$\Delta w = \eta \cdot \delta \cdot x_{entrada}$$

En resumen:

- La **primera fórmula** es la explicación matemática completa (el "por qué").
- La **segunda fórmula** (δ) es el resumen práctico de los dos primeros pasos, que representa la "culpa local" de la neurona antes de mirar sus cables de entrada.

7.7 Resumen del Algoritmo Completo

1. **Inicializar pesos:** Valores pequeños aleatorios (nunca todos a cero).
2. **Repetir** hasta que el error sea bajo:
 - **Forward:** Calcular todas las Net y y desde el inicio hasta el fin.
 - **Cálculo de Error (δ) Salida:** Usando $(t - y)$.
 - **Backpropagation:** Calcular los δ ocultos trayendo el error desde el futuro hacia el pasado usando los pesos.
 - **Update:** Modificar todos los pesos (w) usando la fórmula $\eta \cdot \delta \cdot y$.

7.8 Conclusión de la Asignatura

Una vez más se demuestra que el grado presenta una dejadez considerable por parte de las "vacas sagradas" de la facultad. Y es que no puede ser: estas personas son reconocidísimas en sus ámbitos, publican 300 artículos, acuden a congresos internacionales... pero cuando llega el momento de pensar en sus alumnos y proporcionarles una bibliografía de calidad con la cual puedan aprender los conceptos de forma intuitiva, te sueltan un PDF de 2001 robado de otra universidad o presentaciones sin orden lógico alguno, llenas de palabras sueltas y esquemas incomprensibles.

Eso sí, si te quejas, la alternativa que te ofrecen es el libro de turno de 1990 escrito en alemán por un médico francés. Y apáñate tú para conseguirlo y entenderlo.

La situación es insostenible. Es frecuente encontrarse con:

- Documentación completamente desactualizada
- Materiales de dudosa procedencia sin adaptar al contexto actual
- Presentaciones caóticas sin estructura pedagógica
- Bibliografía obsoleta, inaccesible o en idiomas que nadie domina

¿Tan difícil es hacer las cosas bien? Otras carreras universitarias lo consiguen. No se pide la perfección, pero sí un mínimo de coherencia y actualización en los materiales docentes.

Considero que el grado debería aprender de otras titulaciones y mejorar urgentemente este aspecto. Con la jubilación progresiva de las viejas vacas sagradas de la universidad, espero sinceramente que esta situación mejore con el tiempo y que la nueva generación de profesorado traiga una renovación real en la calidad docente, no solo en la investigadora.

Otra asignatura te lo paso, pero en esta no me jodas. Es imposible que con todos los conocimientos sobre IA que tienes y la cantidad de herramientas que existen, no te salga de dentro redactar unos apuntes propios mediante el uso de la IA.